Laboratorio di programmazione per sistemi ciberfisici

4. Costrutti di iterazione

Enrico Martini *Michele Boldo*

A.A. 2026/2027

Cicli e iterazioni

Un **ciclo** permette di eseguire ripetutamente un gruppo di istruzioni. Ogni esecuzione del corpo del ciclo è un'**iterazione**.

Esempio: stampare la tabellina di 3 fino al moltiplicatore 4.

```
3 x 1 = 3     → iterazione 1
3 x 2 = 6     → iterazione 2
3 x 3 = 9     → iterazione 3
3 x 4 = 12    → iterazione 4
```

A ogni iterazione cambia il moltiplicatore; l'operazione da eseguire è la stessa.

Condizioni e confronti

Una **condizione** è un'espressione valutata come **vera** o **falsa**. Nei cicli determina se continuare la ripetizione.

Gli **operatori di confronto** confrontano due valori:

Operatore	Significato	Esempio
`==`	Uguale a	`n == 3`
`!=`	Diverso da	`n != 0`
`<`	Minore di	`n < 5`
`<=`	Minore o uguale a	`n <= 5`
`>`	Maggiore di	`n > 0`
`>=`	Maggiore o uguale a	`n >= 0`

I confronti restituiscono `1` se veri, `0` se falsi. In una condizione, qualsiasi valore diverso da zero vale come vero.

Attenzione: = assegna, == confronta.

Come si legge una condizione

La condizione serve a decidere **quando continuare** e **quando fermarsi**: vera $\rightarrow$ continua, falsa $\rightarrow$ termina.

Con `n = 3`:

Confronto	Significato	Risultato
`n != 0`	Diverso da zero	vero
`n > 0`	Positivo	vero
`n <= 0`	Negativo o zero	falso
`n == 0`	Uguale a zero	falso

Nella tabellina, `i <= 4` è vera fino a `i = 4`; con `i = 5` il ciclo termina.

Il ciclo while

Sintassi:

```
while ( condizione ){
  corpo del ciclo
}
```

- Se il valore della condizione è vera ($\neq 0$), allora viene eseguito il corpo del ciclo
- Quando la condizione diventa falsa (0), il ciclo termina
- Il corpo del ciclo potrebbe non essere mai eseguito, se la condizione è falsa in partenza
- Il ciclo potrebbe non finire mai se la condizione rimanesse vera per sempre. **Deve essere modificata all'interno del ciclo**

Somma con valore di terminazione

Sommiamo i numeri inseriti dall'utente. Il valore `0`, detto **valore di terminazione**, conclude l'inserimento.

```c
int n, sum = 0;
scanf("%i", &n);        // Prepara il primo controllo

while (n != 0) {
    sum += n;           // Aggiorna la somma
    scanf("%i", &n);    // Prepara il controllo successivo
}

printf("Somma: %i\n", sum);
```

Somma con valore di terminazione: traccia

Input: `3 -2 0` **Stato iniziale:** `sum = 0`

Valore letto	`n != 0`	**Effetto sulla somma**
3	vero	$0 + 3 = \mathbf{3}$
−2	vero	$3 - 2 = \mathbf{1}$
0	falso	Il corpo non viene eseguito

Risultato: 3 letture, 2 iterazioni

Il valore `0` viene letto, ma non sommato.

```
Somma:  1
```

La prima `scanf` legge `3`; quella nel ciclo legge `-2` e poi `0`.
Se il primo input fosse `0`: **nessuna iterazione**, somma `0`.

Il ciclo do-while

Prima esegue il corpo, poi controlla se ripeterlo.

```
1 do {
2     corpo del ciclo
3 } while (condizione);
```

- Il corpo viene eseguito **almeno una volta**.
- La condizione viene valutata **dopo il corpo**: se è vera si ripete, se è falsa si termina.

Attenzione alla sintassi

Dopo `while (condizione)` serve il **punto e virgola**.

Esempio 1: richiedere un numero positivo

La richiesta si ripete finché il numero è negativo o zero.

```c
int n;
do {
    printf("Inserisci un numero positivo: ");
    scanf("%i", &n);
} while (n <= 0);

printf("Il numero %i e' positivo.\n", n);
```

Esempio: `-3` e `0` fanno ripetere la richiesta; `5` termina il ciclo.

Esempio 2: somma con do-while

Sommiamo i numeri inseriti fino al valore di terminazione `0`.

```c
int n, sum = 0;
do {
    scanf("%i", &n);
    sum += n;
} while (n != 0);

printf("Somma: %i\n", sum);
```

Anche lo `0` viene sommato, ma non modifica il risultato.

Se il primo input è `0`, il corpo viene eseguito **una volta**; nell'esempio con `while`, zero volte.

La somma: while e do-while

while

```c
int n, sum = 0;
scanf("%i", &n);
while (n != 0) {
    sum += n;
    scanf("%i", &n);
}
```

Lo `0` viene letto, **non sommato**.

Input `3 -2 0`: **2 iterazioni**.
Input `0`: **0 iterazioni**.

do-while

```c
int n, sum = 0;
do {
    scanf("%i", &n);
    sum += n;
} while (n != 0);
```

Lo `0` viene letto e **sommato**, senza modificare la somma.

Input `3 -2 0`: **3 iterazioni**.
Input `0`: **1 iterazione**.

Il ciclo for

Comodo quando la ripetizione è guidata da un **contatore**.

```
1 for (inizializzazione; condizione; aggiornamento) {
2     corpo del ciclo
3 }
```

- **Inizializzazione**: eseguita una sola volta, prima del primo controllo.
- **Condizione**: controllata prima di ogni iterazione; se è falsa, il ciclo termina.
- **Aggiornamento**: eseguito dopo il corpo, prima del controllo successivo.

Il corpo può essere eseguito **zero volte**, come nel `while`.

Esempio: i quadrati da 1 a n

Leggiamo `n` e stampiamo il quadrato di ogni intero da `1` a `n`.

```c
int n;
printf("Inserisci n: ");
scanf("%i", &n);

for (int i = 1; i <= n; i++) {
    printf("%i - %i\n", i, i * i);
}
```

`i` parte da `1`; `i++` lo aumenta di uno dopo ogni stampa.
Con `n = 3`, stampa le coppie `1 - 1`, `2 - 4`, `3 - 9`. Quando `i` diventa `4`, il ciclo termina.

Esempi di conteggio

Con `n = 3` e un contatore `int i`:

Intervallo	Ciclo	Valori di i
Da 0 a `n-1`	`for (i=0; i<n; i++)`	`0, 1, 2`
Da 1 a `n`	`for (i=1; i<=n; i++)`	`1, 2, 3`
Da `n` a 1	`for (i=n; i>0; i--)`	`3, 2, 1`
Da `n-1` a 0	`for (i=n-1; i>=0; i--)`	`2, 1, 0`

`i++` incrementa il contatore; `i--` lo decrementa.

In tutti e quattro gli esempi il corpo viene eseguito **3 volte**.

Interrompere o saltare: break e continue

break

Termina immediatamente il ciclo.

```c
for (int i=1; i<=5; i++) {
    printf("%i ", i);
    break;
}
```

Output: `1`

Stampa `1`, poi esce dal ciclo.

continue

Salta il resto del corpo.

```c
for (int i=1; i<=5; i++) {
    printf("%i ", i);
    continue;
    printf("X ");
}
```

Output: `1 2 3 4 5`

Salta la stampa di `X`, poi esegue `i++` e ricontrolla la condizione.

Qui le due istruzioni vengono eseguite sempre quando raggiunte. Con `if` potremo decidere **quando** eseguirle.

Equivalenza for - while - do while

while:

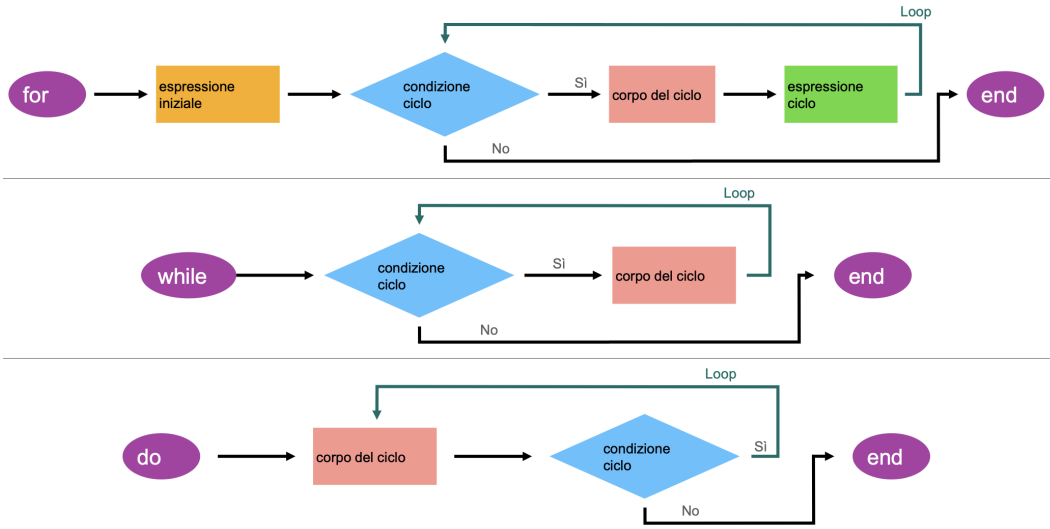
```
espressione iniziale
while( condizione ){
  corpo del ciclo
  espressione di fine corpo
}
```

do while:

```
espressione iniziale
do{
  corpo del ciclo
  espressione di fine corpo
} while( condizione );
```

for:

```
for ( espressione iniziale ; condizione ; espressione di fine corpo ){
        corpo del ciclo
}
```


for
espressione iniziale
condizione ciclo
Sì
corpo del ciclo
espressione ciclo
Loop
end
No
while
condizione ciclo
Sì
corpo del ciclo
Loop
end
No
do
corpo del ciclo
condizione ciclo
Sì
Loop
end
No

Contatore e accumulatore

```
1 int somma = 0;
2 for (int i = 1; i <= 4; i++) {
3     somma += i;
4 }
```

- `i` è il **contatore**: indica il numero da aggiungere.
- `somma` è l'**accumulatore**: conserva il risultato parziale.

$$\texttt{somma:}\quad 0 \xrightarrow{+1} 1 \xrightarrow{+2} 3 \xrightarrow{+3} 6 \xrightarrow{+4} 10$$

Una somma parte da `0`; un prodotto parte da `1`.

L'accumulatore si inizializza prima del ciclo: reimpostarlo nel corpo cancellerebbe il risultato precedente.

Esercizio 1

Scrivere un programma che stampi la tabellina di un numero n inserito dall'utente. La tabellina deve essere calcolata da 1 fino ad un valore y, sempre inserito dall'utente.

Esercizio 2

Scrivi un programma che, dato un intero N positivo inserito dall'utente, ne calcoli il numero triangolare (somma dei numeri da 1 a N).

Esercizio 3

Scrivi un programma che, dato un intero N positivo inserito dall'utente, ne calcoli il fattoriale (prodotto dei numeri da 1 a N).

Esercizio 4

Scrivere un programma che conta le crifre di un intero, inserito dall'utente.

Esercizio 5

Scrivere un programma che data base ed esponente ne calcoli la potenza.

Esercizio 6

Scrivere un programma che inverta le cifre di un numero intero inserito dall'utente.

Esercizio 7

Scrivere un programma che stampi il calendario di un mese, dati in input il numero di giorni e il giorno della settimana in cui comincia (1=lun, 7=dom).

Esercizio 8

Scrivere un programma che approssimi il valore di e (con n intero positivo immesso dall'utente):

$$e = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \frac{1}{4!} + ... + \frac{1}{n!} \quad (1)$$

Esercizio 9

Scrivi un programma che chieda all'utente un numero intero n e stampi la serie di fibonacci dei primi n numeri. La serie di Fibonacci inizia con 0 e 1, e ogni numero successivo è la somma dei due precedenti (1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89,144).

Esercizio 10

Scrivere un Programma C che stampi una matrice n×n, dove n è un valore richiesto all'utente. In ogni cella della matrice va stampata la coppia (i, j) dove i è il numero di riga e j il numero di colonna.

Suggerimento: Utilizzare opportunamente le stringhe di formato (`%2i`) *per impostare la larghezza di campo. Assumere che n sia composto al massimo da 2 cifre.*

```
Inserire un numero intero: 4
( 1, 1) ( 1, 2) ( 1, 3) ( 1, 4)
( 2, 1) ( 2, 2) ( 2, 3) ( 2, 4)
( 3, 1) ( 3, 2) ( 3, 3) ( 3, 4)
( 4, 1) ( 4, 2) ( 4, 3) ( 4, 4)
```

Esercizio 11

Scrivere un programma che calcoli la serie armonica fino al n-esimo termine, controllando che n sia maggiore di 1.

$$r = \sum_{k=1}^{n} \frac{1}{k}$$

Esercizio 12

Scrivere un programma per il calcolo del coefficiente binomiale, definito come:

$$\binom{a}{n} = \prod_{k=1}^{n} \frac{a-k+1}{k}$$

dove $a, n \in N$ sono inseriti dall'utente, con $n < a$.